中山大學
SUN YAT-SEN UNIVERSITY



指针与结构体应用进阶

中山大学计算机学院

讲课人：郑贵锋

目录

CONTENTS

01 Stack数据结构

02 Stack的链表实现

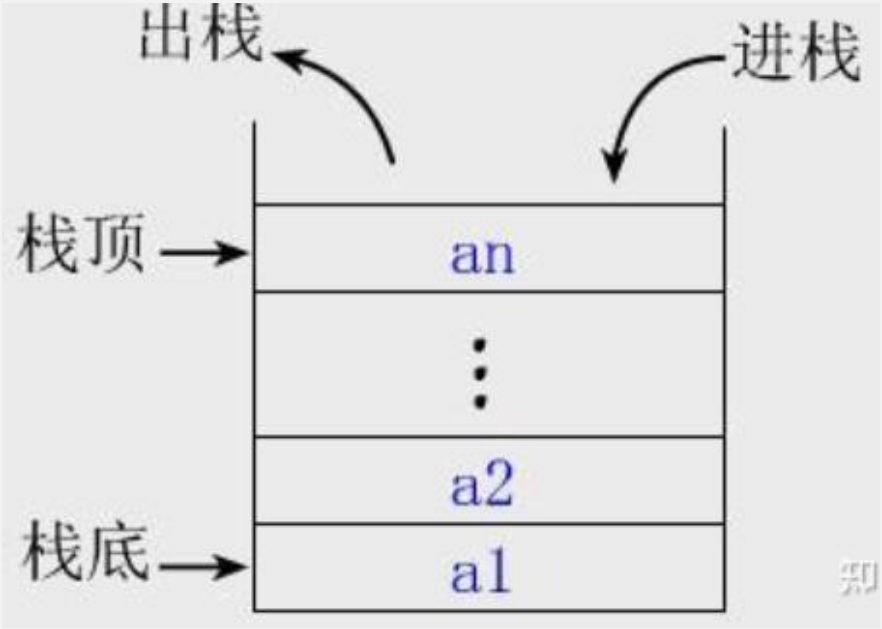
03 Stack的数组实现

04 Stack应用：有效的括号

05 共用体

Stack 数据结构及应用

Stack（栈）是一种线性的数据结构，因此它既可以用数组实现，也可以用链表实现。栈的特征是只允许在一端进行插入或删除操作。



Stack 数据结构及应用

栈的基本操作

1） void InitStack(Stack* s)：初始化空栈S
2） int StackEmpty(const Stack* s)：判断一个栈是否为空
3） void Push(Stack* s,const int x)：进栈，将x加入使之成为新栈顶
4） int Pop(Stack* s)：出栈，若栈非空，则将栈顶元素返回
5） void DestroyStack(Stack* s)：销毁栈，并释放栈S占用的存储空间

（1）Stack 数据结构——链表实现

根据链表实现，可以完成栈的基本操作（详细代码参见stack_list.c）
首先我们只需要记录一个栈顶节点即可，因此定义如下

```
typedef struct StackList{
        NODE_PTR head;
}StackList;
void Push(StackList* s, const int x){//入栈一个元素
        s->head=ListAdd(s->head, x);
}
int Pop(StackList* s){//出栈一个元素
        assert(s->head);//检查是否为空，对空栈进行出栈操作是一个错误
        int t=s->head->customerData;
        s->head=ListRemove(s->head);
        return t;
}
```

（2）Stack 数据结构——数组实现

使用数组也可以实现栈，由于数组是固定大小的，因此需要记录当前栈元素数量，以避免溢出。（详细代码参见stack_array.c）

```
typedef struct StackArray{
        int* data;
        int cnt;//记录目前栈内有多少元素
        size_t max_size;//记录栈最多可以容纳多少元素
}StackArray;

void InitStack(StackArray* s,size_t size){//初始化栈
        s->data=(int*)malloc(size*sizeof(int));
        s->max_size=size;
        s->cnt=0;
}
```

（3）Stack 数据结构应用——有效的括号

给定一个只包括 `'('`，`')'`，`'{'`，`'}'`，`'['`，`']'` 的字符串 `s`，判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

求解方法：从左到右扫描字符串，若遇到左括号入栈，遇到右括号时将栈顶的对应左括号出栈。若遇到右括号时栈顶的左括号与之不匹配，则无效；若到结束时栈内还有元素，则无效；其余情况皆为有效字符串。（详细代码参见valid_str.c）

例如：“{[({})]}” 有效
“{[({])]}” 无效

（3）Stack 数据结构应用——有效的括号

```
for(i=0;str[i]!='\0';i++){//从左到右扫描
    if(str[i]=='{' || str[i]=='(' || str[i]=='['){
        Push(&s,str[i]);//左括号直接入栈
    }else{//遇到右括号，则匹配对应括号出栈
        if(StackEmpty(&s)){//没有括号匹配，则字符串无效
            printf("false\n");
            return 0;
        }
        char c=Pop(&s);
        if((str[i]=='}' && c=='{') || (str[i]==')' && c=='(') || (str[i]==']' && c=='[')){
            continue;//括号匹配了，则继续
        }else{//括号不匹配，则字符串无效
            printf("false\n");
            return 0;
        }
    }
}
if(!StackEmpty(&s)){//最后栈非空则无效
    printf("false\n");
}else{
    printf("true\n");
}
```

(4) 共用体（Union）

共用体与结构体一样是一种派生数据类型。在共用体中，成员共享一个存储空间。

```
union number{
        int x;
        double y;
};
```

例如number这个共用体中，x和y是共用一个存储空间的。共用体所占用的空间取决于对齐值。

(4) 共用体（Union）

一个共用体使用的例子如下（union.c）

```
#include<stdio.h>
union number{
        int x;
        double y;
};
int main(){
        union number value;
        value.x=100;
        printf("value.int:%d, value.double:%lf\n",value.x,value.y);
        value.y=100;
        printf("value.int:%d, value.double:%lf\n",value.x,value.y);
        printf("sizeof value:%d\n",sizeof(value));
}
```

问题：如果先给value.y赋值，再给value.x赋值，结果一样吗？

（Gdb命令查看64位二进制：x/1tg &value）

（4）共用体（Union） - 练习

IP地址的表示。

IP 地址是一个 unsigned long int，用于各种地址相关运算。出于可读性和便于记忆，通常用4个0-255的整数显示、编辑。例如：某机器 IP 地址是 172.6.6.24

- 请给出 IP Address 的数据类型定义
- 初始化一个 IP Address 的实例。其值为 172.6.6.24

中山大學
SUN YAT-SEN UNIVERSITY



谢 谢

中山大学计算机学院

编制人：课题组